About

This lab lets you practice implementing your own functions in Python as well as using the standard List methods. By the end of this assignment, you will have developed a simple To-do list using Python.

Learning Objectives

1. Implement simple functions in Python.
2. Use a list as a data type to store multiple elements.
3. Use Python's built-in List methods to implement simple functions.

Estimated Size

1 program file.

Setup

In your Python files, you must include author information. On the first line, add a `# Author` comment line, where you mention your full name. Similarly, include your email and Spire ID on the `# Email` and `# Spire ID` comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author   : John Snow
# Email    : jsnow@umass.edu
# Spire ID : 12345678
```

Download the starter code file `to_do_list.py` and open it in VSCode. **All your work** should be in this single file, and this is the file you will submit to Gradescope in the end. First, please **modify the comments section at the top of the file** to include your information.

0. Data type and `clear_tasks`

A to-do list contains descriptions of tasks that you need to complete. A to-do list can be modified so you can add new tasks and delete completed tasks. For this reason, the to-do list must be mutable. So we will use the List data type to represent it. In the following, we will use the variable name `to_do_list` to represent the list object. In Python, we can use `to_do_list = []` to create an empty list and assign it to the variable named `to_do_list`.

As a starting example, the function `clear_tasks` has been provided to you in the starter code. This function takes a list object as a parameter and calls its `clear()` method to delete all tasks and make the list empty. You do NOT need to change this function.

1. Implement `add_task` (5 points)

A to-do list should allow the user to add a new task. The first exercise is to implement a function called

`add_task`. This function should take **two parameters**: a list object representing the to-do list, and a string object representing (the description of) a task. Your function should

- Add that task **to the end of the list**.
- Return an acknowledgement, which should be **a string** formatted exactly as:
  `Task successfully added. N tasks remaining.`
  where `N` is the total number of tasks currently in the to-do list. Your returned string must match the format above exactly -- pay attention to the space characters. Do not include any extra white space characters.

Assuming you have implemented `add_task` correctly, below are some examples of calling the function and the expected return values. The starter code contains some example code to help you **test your function**. Follow the instructions there. You can feel free to add more tests to help verify your work.

```
print(add_task(to_do_list, 'zybook reading'))
Task successfully added. 1 tasks remaining.

print(add_task(to_do_list, 'do laundry'))
Task successfully added. 2 tasks remaining.

print(add_task(to_do_list, 'cics110 lab 3'))
Task successfully added. 3 tasks remaining.
```

2. Implement `delete_task` (5 points)

A to-do list should also allow the user to delete a task once it's completed. So your second exercise is to implement a function called `delete_task`. This function should take **two parameters**: a list object representing the to-do list, and a string object representing (the description of) a task to be deleted. Your function should:

- Delete that task from the list. You can assume that the task exists in the to-do list, and also that there are no duplicates in the to-do list.
- Return an acknowledgement, which should be **a string** formatted exactly as:
  `Task successfully deleted. N tasks remaining.`
  where `N` is the number of tasks in the to-do list **after the removal**.

Below are some examples of calling the function and the expected return values. Again, your returned string must be formatted exactly as requested above.

```
print(delete_task(to_do_list, 'do laundry'))
Task successfully deleted. 2 tasks remaining.

print(delete_task(to_do_list, 'cics110 lab 3'))
Task successfully deleted. 1 tasks remaining.
```

3. Implement `move_task` (5 points)

Sometimes we want to reorder items in the to-do list so more important or urgent tasks will be moved up and less important ones will be moved down. Here, you will implement a function called `move_task`. This function should take **three parameters** -- a list object representing the to-do list, the index of a to-do item that you want to move (we can call this the `from_index`), and an index where you want it to be moved to (call this the `to_index`). Please note that in Python, the index of an item in a List starts from 0: i.e., the first item is at index 0, the second is at index 1, and so on. Your function should:

- Move the to-do list item currently at the `from_index` to the `to_index`, and ordering of all other items should remain the same as before. You can assume both `from_index` and `to_index` are valid, so there will NOT be an out of range error.
- Return an acknowledgement, which should be **a string** formatted exactly as:
  `Task 'T' successfully moved to index J`
  where `T` is the description of the task surrounded by **single quotes** `'` and `J` is the index it has been moved to. *Note there is no dot at the end of the printout.*

Assuming you have implemented `move_task` correctly, below are some examples of calling the function and the expected return values.

```
to_do_list = ['zybook reading', 'do laundry', 'cics110 lab 3']

print(move_task(to_do_list, 2, 0))
Task 'cics110 lab 3' successfully moved to index 0

print(to_do_list)
['cics110 lab 3', 'zybook reading', 'do laundry']

print(move_task(to_do_list, 1, 2))
Task 'zybook reading' successfully moved to index 2

print(to_do_list)
['cics110 lab 3', 'do laundry', 'zybook reading']
```

4. Submit your code to Gradescope

Log in to Gradescope, select **Lab 02: (Code)**, and submit **`to_do_list.py`**. Wait until the autograder finishes running and returns the result to you. You can resubmit as many times as you like before the deadline.

5. Submit Lab 2 Questionnaire on Gradescope

Remember to submit the Lab 2 questionnaire to Gradescope too.

If you see a question you do not know how to answer, ask TAs/UCAs in your lab, or use Piazza, or go to

one of our office hours.

(Common Paragraph) Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.

You may not use any code that is posted on the internet. If you are not sure, it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the course as well as online resources. It is very easy for us to detect similar submissions, which will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the UMass Academic Honesty Policy and Procedures so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty, you should re-read the course policies. We assume you have read the course policies in detail, and by submitting this project, you have provided your virtual signature in agreement with these policies.